

Milestone 3. route.py Development

Activity 3.1. Writing the Main Application Logic in routes.py

Milestone 3. Routes.py Development

Milestone 3 focuses on building the routing logic inside the routes.py file, which acts as the core operational layer of the FitBuddy application. It bridges the frontend templates, AI generation modules (Gemini Pro and Flash), and the SQLite database. Each route defines how user inputs are collected, processed, and responded to — creating a dynamic, personalized experience for fitness planning and feedback-based updates.

Activity 3.1: Writing the Main Application Logic in routes.py

Defining Core routes in routes.py:

Separate routes were implemented to serve each of FitBuddy's main functionalities:

/ : Displays the user form via index.html

/generate-workout: Processes user input and generates a personalized workout plan

/submit-feedback: Updates the workout plan based on user feedback

/view-all-users: Displays an admin dashboard showing all users and their plans

Set up Route Handlers for Each Feature

► / – Home Route

Returns the homepage (index.html)

No form processing logic

► /generate-workout – Plan Generator

Receives user input using FastAPI's Form(...)

Constructs a UserInput Pydantic model

Calls:

Generate_workout_gemini(...) → Generates workout plan via Gemini 1.5 Pro

Generate_nutrition_tip_with_flash(...) → Gets nutrition tip via Gemini Flash

Stores:

User details with save_user(...)

Plan with save_plan(...)

Returns result.html with dynamic data

► /submit-feedback – Update Plan with Feedback

Captures feedback and user_id

Retrieves original plan from DB

Uses update_workout_plan(...) to revise the plan via Gemini 1.5 Pro

Updates the revised plan in the DB using update_plan(...)

Renders updated result on result.html

► /view-all-users – Admin View

Calls get_all_users() and get_all_plans() to fetch DB records

Passes them to all_users.html

Admin can view:

Username, Age, Weight, Goal, Intensity

Original and updated workout plans

Integrate Gemini AI Responses in Each Function

Each AI-related route calls Google's Gemini models with specific prompts:

Workout Plan Generation:

Generate_workout_gemini() sends fitness goal and intensity to Gemini Pro to return a structured 7-day workout plan.

Nutrition Tip Generation:

Generate_nutrition_tip_with_flash() uses Gemini Flash to generate a practical tip related to the user's fitness goal.

Workout Plan Update via Feedback:

Update_workout_plan() passes original plan + user feedback to Gemini Pro and returns a revised plan.

All responses are returned to the user through result.html, maintaining clarity using <pre> blocks for formatting.

Workout generation route Nutrition/Recovery tip generation route

1. API: Generate workout using Gemini Pro

```
@router.post("/generate-workout/gemini")
```

```
async def generate_gemini_workout(request: WorkoutRequest):
```

```
    try:
```

```
        result = generate_workout_gemini({
```

```
            "goal": request.goal,
```

```
            "intensity": request.intensity
```

```
        })
```

```
        return {"model": "gemini-pro", "workout_plan": result}
```

```
    except Exception as e:
```

```
        raise HTTPException(status_code=500, detail=str(e))
```

2. API: Generate nutrition tip using Gemini Flash

```
@router.get("/nutrition-tip")
```

```
def get_flash_tip(goal: str):
```

```
    tip = generate_nutrition_tip_with_flash(goal)
```

```
    return {"goal": goal, "nutrition_tip": tip}
```

3. API: Save user info & generate plan

```
@router.post("/generate-plan")
```

```
def generate_plan(user_data: UserInput):  
    try:  
        save_user(  
            user_id=user_data.user_id,  
            name=user_data.username,  
            age=user_data.age,  
            weight=user_data.weight,  
            goal=user_data.goal,  
            intensity=user_data.intensity  
        )  
  
        plan = generate_workout_gemini({  
            "goal": user_data.goal,  
            "intensity": user_data.intensity  
        })  
  
        save_plan(user_data.user_id, plan)  
  
        return {  
            "message": "Workout plan generated and saved successfully!",  
            "workout_plan": plan  
        }  
  
    except Exception as e:  
        raise HTTPException(  
            status_code=500,
```

```
        detail=f"Something went wrong: {str(e)}"  
    )
```

4. API: Update workout plan based on user feedback

```
@router.post("/update-plan/{user_id}", response_model=dict)
```

```
def update_user_plan(user_id: int, data: FeedbackRequest):
```

```
    original = get_original_plan(user_id)
```

```
    if not original:
```

```
        return {"error": "Original plan not found for this user."}
```

```
    updated = update_workout_plan(original, data.feedback)
```

```
    update_plan(user_id, updated)
```

```
    return {"updated_plan": updated}
```